SQL Documentation

Course: DLBDSPBDM01 – Build a Data Mart in SQL; **Programme:** B.Sc. Computer Science; **Author:** Zubair Ahmed Khan; **Matriculation Number:** 92127983; **Tutor:** Prof. Dr. Anna Androvitsanea; **Submission Date:** 2026-07-29

---

1. Purpose

This document accompanies the SQL file `airbnb.sql` and explains how the script is organised, the order in which the statements must be executed, the dependencies between them, and the way in which tables, relationships, and dummy data are created. The aim is to make the script readable, auditable, and reproducible for academic grading and for future maintenance.

2. File Overview

Property	Value
File name	`airbnb.sql`
File size	Approximately 20 KB
Total SQL statements	70+ (mix of DDL and DML)
Number of tables	21
Database engine	MySQL 8.0
Character set	Default (`utf8mb4`)
Storage engine	Default InnoDB

The script is a single self-contained file. It does not depend on any external file, dump, or schema object outside its own statements.

3. High-Level Structure

The file is organised into the following five logical sections, in execution order:

1. **Section A — Database bootstrap.** `SHOW DATABASES;`, `CREATE DATABASE AirbnbDB;`, `USE AirbnbDB;`, `SELECT DATABASE();`.
2. **Section B — First pass of core tables and seed data.** `User`, `Guest`, `Host`, `Accommodation`, and `Booking`, followed by a small initial seed.
3. **Section C — Creation of remaining tables.** `Payment`, `Reservation`, `Review`, `Complaint`, `Notification`, `Location`, `HouseRules`, `Place`, `Amenity`, `AmenityListing`, `Media`, `Listing`, `Employee`, `Language`, `UserLanguage`, `Income`.
4. **Section D — Schema evolution through `ALTER TABLE`.** Additional columns and foreign keys are added to `Accommodation`, `Booking`, and `Payment`.
5. **Section E — Bulk data insertion.** Each remaining table receives at least twenty records.

The sections must be executed from top to bottom. Reordering is not supported, because each section depends on objects created in the previous one.

4. Execution Order and Dependencies

The dependency graph between tables is summarised below. An arrow `A → B` means that table `B` references a primary key in table `A`, so `A` must be created and populated before `B`.

```
User → Guest
User → Host
User → Review (ReviewerID, ReviewedID)
User → Notification
User → UserLanguage
Language → UserLanguage
Host → Accommodation
Host → Complaint (RelatedHostID)
Guest → Complaint (RelatedGuestID)
Guest → Booking
Location → Place
HouseRules→ Place
Place → Accommodation
Place → AmenityListing
Amenity → AmenityListing
Accommodation → Booking
Booking → Payment
Booking → Reservation
Payment → Reservation
Media → Listing
Place → Listing
Host → Income
Employee → Employee (recursive ManagerID)
```

Because the script uses **forward references** (a table is created in a section, populated in a later section, and finally joined by another table that is created even later), the `ALTER TABLE` statements in Section D are the natural way to add foreign keys that could not have been declared during the initial `CREATE TABLE` because the referenced table did not yet exist. The two `ALTER TABLE Accommodation` blocks add the `Description`, `Photos`, `Availability`, and `PlaceID`

columns, with `PlaceID` carrying the foreign key back to `Place`. A separate `ALTER TABLE Payment` adds the `PaymentDate` column after the initial insert.

5. Detailed Section Walk-Through

5.1 Section A — Database Bootstrap

The first statement inspects the server, the second creates the database, the third switches the active schema, and the fourth confirms the switch. From this point on, every unqualified table name resolves to `AirbnbDB.<table>`.

5.2 Section B — Core Tables and Initial Seed

The first five tables form the backbone of the schema:

- `User(UserID, Name, Email, PhoneNumber, ProfilePicture, UserType)` — the platform participant.
- `Guest(GuestID, UserID)` — specialisation of a `User` who books stays.
- `Host(HostID, UserID)` — specialisation of a `User` who lists accommodations.
- `Accommodation(AccommodationID, HostID, Title, Location, PricePerNight)` — a property listed by a host.
- `Booking(BookingID, GuestID, AccommodationID, CheckInDate, CheckOutDate, TotalPrice)` — a reservation of an accommodation by a guest.

Each `CREATE TABLE` is followed by a `SHOW TABLES;` for visual confirmation. Two seed rows per table are inserted at this stage to allow the test cases to return data immediately after the schema is created.

5.3 Section C — Supporting Tables

The remaining tables are created next, in roughly the order in which they appear in the file:

- `Payment` and `Reservation` extend the booking lifecycle.
- `Review`, `Complaint`, and `Notification` capture user feedback and platform messaging.
- `Location`, `HouseRules`, `Place`, `Amenity`, `AmenityListing`, `Media`, and `Listing` describe the physical side of an accommodation.
- `Employee`, `Language`, `UserLanguage`, and `Income` cover supporting concerns (organisational hierarchy, multilingual users, and financial tracking).

This order respects the foreign-key graph; for example, `AmenityListing` is created after both `Place` and `Amenity`, and `Reservation` is created after both `Booking` and `Payment`.

5.4 Section D — Schema Evolution

Three `ALTER TABLE` statements add the columns and constraints that depend on tables created later in the file:

```
<code class="language-sql">ALTER TABLE Accommodation
```

```
ADD Description VARCHAR(255),
ADD Photos BLOB,
ADD Availability DATETIME;

ALTER TABLE Booking
ADD PaymentStatus VARCHAR(50),
ADD TransactionDetails VARCHAR(255);

ALTER TABLE Accommodation
ADD PlaceID INT,
ADD FOREIGN KEY (PlaceID) REFERENCES Place(PlaceID),
CHANGE PricePerNight Pricing DECIMAL(10,2);
```

The final `ALTER TABLE` renames `PricePerNight` to `Pricing` and adds the foreign key to `Place`. Note that the third `ALTER` introduces a deliberate schema refinement: the price column is renamed to make the model more idiomatic while keeping the data intact.

A later `ALTER TABLE Payment ADD PaymentDate DATE;` adds the missing temporal column after the payment records have been inserted, allowing the script to remain linear.

5.5 Section E — Bulk Data Insertion

Each table receives at least twenty rows of dummy data. The `INSERT` statements use the explicit column form (`INSERT INTO Table(col1, col2, …) VALUES (…)`) so that they remain valid even after the `ALTER TABLE` blocks have added or renamed columns. Each insertion is followed by a `SELECT COUNT(*) FROM <table>;` so that the operator can verify the row count at runtime.

The dummy data is realistic in the sense that it uses plausible names, Indian and international locations, valid date ranges, and positive monetary amounts. It is, however, fictitious and safe to publish.

6. Constraints and Data Types

The following conventions are applied consistently across the file:

Concern	Convention
Primary key naming	`<TableName>ID`, integer, auto-incrementable in spirit (assigned manually)
Foreign-key naming	`<ReferencedTable>ID`
Monetary values	`DECIMAL(10,2)`
Date values	`DATE` for day-level precision, `DATETIME` for timestamps

Concern	Convention
Short text	`VARCHAR(50)` to `VARCHAR(255)`
Binary large objects	`BLOB` (used for `ProfilePicture` and `Photos`)
Booleans	Implicit through `Status` columns such as `PaymentStatus`
Character set	Server default (`utf8mb4`)
Storage engine	InnoDB (default) — required for foreign-key enforcement

7. Verification Queries

After the script has been executed, the following statements can be used to verify the installation:

```
<code class="language-sql">-- Number of tables
SELECT COUNT(*) FROM information_schema.tables
WHERE table_schema = 'AirbnbDB';

-- Foreign-key count
SELECT COUNT(*) FROM information_schema.referential_constraints
WHERE constraint_schema = 'AirbnbDB';

-- Row count per table
SELECT table_name, table_rows
FROM information_schema.tables
WHERE table_schema = 'AirbnbDB'
ORDER BY table_name;
```

The first statement should return `21`, the second a value greater than `15`, and the third a result set in which every row count is at least `20`.

8. Re-Execution and Idempotency

The script is **not** idempotent. It is designed to be run on a freshly created database. To re-run the entire file, drop the existing database first:

```
<code class="language-sql">DROP DATABASE IF EXISTS AirbnbDB;
```

Then re-execute the script from the top.

9. Extension Points

The script is a coursework artefact, but the schema is sufficiently clean to support future work. The following extensions are realistic and would not require structural changes:

- Adding an index on `Booking(GuestID)` or `Accommodation(HostID)` to accelerate joins.
- Introducing a `Currency` table and converting the `Pricing` and `TotalPrice` columns to foreign keys.
- Adding a stored procedure that books an accommodation atomically, creating a `Booking`, a `Payment`, and a `Reservation` in a single transaction.
- Adding audit columns (`CreatedAt`, `UpdatedAt`) to each table to support change tracking.
- Building a thin REST layer on top of the three test-case queries for use in a reporting dashboard.

10. Summary

The SQL file is a single, self-contained script that builds the entire AirbnbDB data mart in five logical steps. Foreign-key constraints are enforced both at `CREATE TABLE` time and through targeted `ALTER TABLE` statements. Dummy data is provided for every table in sufficient volume to support realistic reporting queries. The three test cases described in the test-case document exercise the schema through multi-table joins and confirm that the data mart is internally consistent.